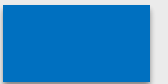




Chapter Five

Cascading Style Sheet (CSS)



Outline

CSS Basics

- ✓ Introduction to CSS
- ✓ CSS Syntax (CSS Selectors and Declarations)
- ✓ Attaching CSS with HTML (External , Embedded and Inline)

Style Sheet Rules

- ✓ Style Inheritance
- ✓ Style Rules Precedence

Style Properties

- ✓ Foreground and Background Properties
- ✓ Font and Text Properties
- ✓ CSS Box Model
- ✓ Table Styling Properties
- ✓ More On Styling List (Creating Navigation bars)
- ✓ Layout and Positioning Properties



What is CSS

- ❖ It is a simple design language intended to simplify the process of making web pages presentable.
- ❖ CSS handles the look and feel part of a web page.
- ❖ CSS is easy to learn and understand but it provides powerful control over the presentation of an HTML document.
- ❖ Most commonly, CSS is combined with the markup languages HTML or XHTML.

What is CSS

❖ Cascading Style Sheets (CSS)

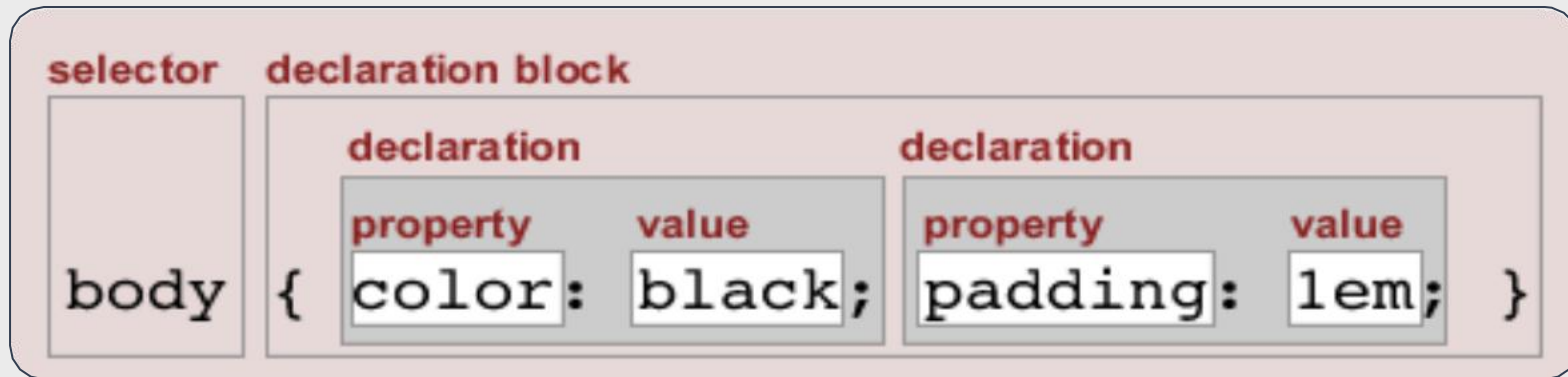
- ✓ Used to describe the arrangement of documents
- ✓ Define sizes, spacing, fonts, colors, layout, etc.
- ✓ Improve content accessibility
- ✓ Improve flexibility

CSS Syntax

- ❖ Some CSS styles are **inherited** and some not
 - ✓ Text-related properties are inherited font-size, font-family, line-height, text-align, list-style, etc
 - ✓ Box-related and positioning styles are not inherited - width, height, border, margin, padding, position, float, etc.

Style Sheets Syntax

- ❖ Stylesheets consist of rules, selectors, declarations, properties and values



- ❖ Selectors are separated by comma
- ❖ Declarations are separated by semicolons
- ❖ Properties and values are separated by colons

```
h1,h2,h3 { color: green; font-weight: bold; }
}
```

Selectors

❖ Selectors used to determine *on which element the rule will be applied*:

- ✓ All elements of specific type (tag)
- ✓ Those that match a specific attribute (id, class)
- ✓ Elements may be matched depending on *how they are nested* in the document tree (HTML)

• Examples:

```
.header a { color: green }
```

```
#menu>li { padding-top: 8px }
```

Selectors (2)

❖ Three primary kinds of selectors:

- By tag (type selector):

```
h1 { font-family: verdana,sans-serif; }
```

- By element id:

```
#element_id { color: #ff0000; }
```

- By element class name (only for HTML):

```
.myClass {border: 1px solid red}
```

❖ Selectors can be combined with commas:

```
h1, .link, #top-link {font-weight: bold}
```

This will match <h1> tags, elements with class link, and element with id top-link

Values in the CSS Rules

❖ Colors are set in RGB format (decimal or hex):

- Example: `#a0a6aa = rgb(160, 166, 170)`
- Predefined color : black, blue, etc.

❖ Numeric values are specified in:

- Pixels, e.g. 12px ,
- Points, inches, centimeters, millimeters
 - E.g. 10pt , 1in, 1cm, 1mm
- Percentages, e.g. 50%
- Zero can be used with no unit: `border: 0;`

Linking HTML and CSS

❖ HTML (content) and CSS (presentation) can be linked in three ways:

➤ **Inline**: the CSS rules in the style attribute

✓ No selectors are needed

➤ **Embedded**: in the <head> in a <**style**> tag

➤ **External**: CSS rules in separate file (best)

✓ Usually a file with .css extension

✓ Linked via <**link** rel="stylesheet" href=...> tag or @import directive in embedded CSS block

Inline Styles:

inline-styles.html Example

```
<!DOCTYPE html">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
  <title>Inline Styles</title>
</head>
<body>
  <p>Here is some text</p>
  <!--Separate multiple styles with a semicolon-->
  <p style="font-size: 30pt">Here is some
    more text</p>
  <p style="font-size: 20pt;color:
    #0000FF" >Even more text</p>
</body>
</html>
```

Here is some more text

Even more text

Embedded Styles

- Embedded in the HTML in the `<style>` tag:
 - The `<style>` tag is placed in the `<head>` section of the document
- Used for document-specific styles

```
<style>  
...  
</style>
```

Embedded Styles: Example

```
<!DOCTYPE html>
<html>
<head>
<style>
body {background-color: powderblue;}
h1  {color: blue;}
p   {color: red;}
</style>
</head>
<body>
<h1>This is a heading</h1>
<p>This is a paragraph.</p>
</body>
</html>
```

This is a heading

This is a paragraph.

External CSS Styles

❖ External linking

- Separate pages can all use a shared style sheet
- Only modify a single file to change the styles across your entire Web site (see <http://www.csszengarden.com/>)

❖ link tag (with a rel attribute)

- Specifies a relationship between current document and another document

```
<link rel="stylesheet" type="text/css" href="styles.css">
```

link elements should be in the <head>

External Styles: Example (2)

external-styles.html

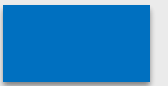
styles.css

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
<h1>This is a heading</h1>
<p>This is a paragraph.</p>
</body>
</html>
```

```
body
{
  background-color: pink;
}
h1 {
  color: blue;
}
p {
  color: red;
}
```

Introduction to CSS Inheritance

- ❖ Inheritance is one feature that leverages the ability to use a styling feature of **the parent entity in a child entity**.
- ❖ This gives ease to the coding, as a common property **need not be styled twice or multiple times**.
- ❖ The use of '**inherit**' keyword makes sure that the property is adopted from the parent entity to the child entity.



The syntax :

```
.Parent{  
  property1: value1;  
  property2:value2;  
}  
.child{  
  property1: inherit;  
  propert2:value3;  
}
```

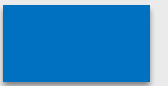
Style Rules

Precedence

- ❖ Every CSS selector has its place in the specificity hierarchy.
- ❖ There are four categories which define the specificity level of a selector:
 1. **Inline styles** - Example: `<h1 style="color: pink;">`
 2. **IDs** - Example: `#navbar`
 3. **Classes, pseudo-classes, attribute selectors** - Example: `.test`, `:hover`, `[href]`
 4. **Elements and pseudo-elements** - Example: `h1`, `:before`

How to Calculate Specificity?

- ❖ Start at 0, add 100 for each **ID** value, add 10 for each class value (or pseudo-class or attribute selector), add 1 for each element selector or pseudo-element.
- ❖ Note: Inline style gets a specificity value of 1000, and is always given the highest priority!
- ❖ Note 2: There is one exception to this rule: if you use the !important rule, it will even override inline styles!



examples on how to calculate specificity values:

Selector	Specificity Value	Calculation
p	1	1
p.test	11	1 + 10
p#demo	101	1 + 100
<p style="color: pink;">	1000	1000
#demo	100	100
.test	10	10
p.test1.test2	21	1 + 10 + 10
#navbar p#demo	201	100 + 1 + 100
*	0	0 (the universal selector is ignored)

Cont..

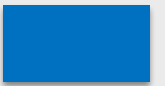
- Example

A: h1

B: h1#content

C: <h1 id="content" style="color: pink;">Heading</h1>

- ❖ The specificity of A is 1 (one element selector)
- ❖ The specificity of B is 101 (one ID reference + one element selector)
- ❖ The specificity of C is 1000 (inline styling)
- ❖ Since the third rule (C) has the highest **specificity value (1000)**, this style declaration will be applied.



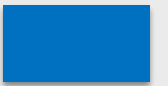
!important?

```
<style?
#myid { background-color: blue;}
.myclass { background-color: gray;}
  p { background-color: pink !important;}
</style>
<body>
<p>This is some text in a paragraph.</p>
<p class="myclass">This is some text in a
paragraph.</p>
<p id="myid">This is some text in a paragraph.</p>
</body>
```

This is some text in a paragraph.

This is some text in a paragraph.

This is some text in a paragraph.



Text-related CSS Properties

- ❖ color - specifies the color of the text
- ❖ font-size - size of font: xx-small, x-small, small, medium, large, x-large, xx-large, smaller, larger or numeric value
- ❖ font-family - comma separated font names
 - Example: verdana, sans-serif, etc.
 - There should always be at least one generic font
- ❖ font-weight can be normal, bold, bolder, lighter or a number in range [100 ... 900]

CSS Rules for Fonts

(2)

- font-style
 - Values: normal, italic, oblique
- text-decoration
 - Values: none, underline, line-through, overline, blink
- text-align
 - Values: left, right, center, justify

Shorthand Font Property

- font
 - Shorthand rule for setting multiple font properties at the same time

```
font:italic normal bold  
12px/16px verdana
```

- is equal to writing this:

```
font-style: italic;  
font-variant: normal;  
font-weight: bold;  
font-size: 12px;  
line-height: 16px;  
font-family: verdana;
```

Backgrounds

- background-image
 - URL of image to be used as background, e.g.:

```
background-image:url("back.gif");
```

- background-color
 - Using color
- background-repeat
 - repeat-x, repeat-y, repeat, no-repeat
- background-attachment
 - fixed / scroll

Backgrounds (2)

- ❖ Background-position: specifies vertical and horizontal position of the background image
 - Vertical position: top, center, bottom
 - Horizontal position: left, center, right
 - Both can be specified in percentage or other numerical values
 - Examples:

```
background-position: top left;
```

```
background-position: -5px 50%;
```

Borders

- ❖ `border-width`: thin, medium, thick or numerical value (e.g. 10px)
- ❖ `border-color`: color alias or RGB value
- ❖ `border-style`: none, hidden, dotted, dashed, solid, double, groove, ridge, inset, outset
- ❖ Each property can be defined separately for left, top, bottom and right
 - `border-top-style`, `border-left-color`, ...

Width and Height

- ❖ width - defines numerical value for the width of element,
e.g. 200px
- ❖ height - defines numerical value for the height of element,
e.g. 100px
 - By default the height of an element is defined by its content
 - Inline elements do not apply height, unless you change their display style.

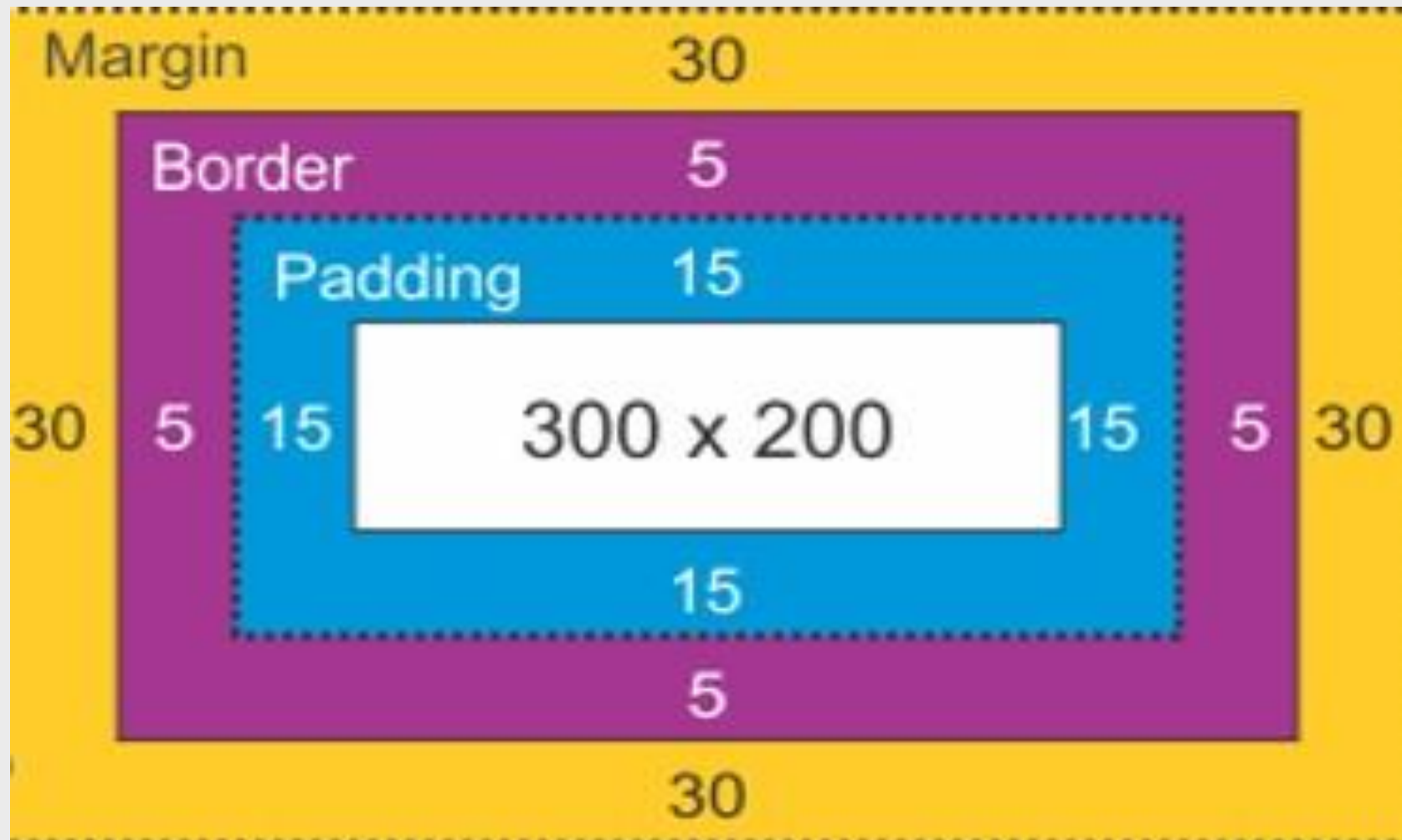
Margin and Padding

- ❖ margin and padding define the spacing around the element
 - ✓ margin is the **spacing outside of the border**
 - ✓ padding is the spacing **between the border and the content**
 - ✓ Numerical value, e.g. 10px or -5px
 - ✓ Can be defined for each of the four sides separately - margin-top, padding-left, ...

Margin and Padding: Short Rules

- `margin: 5px;`
 - ✓ Sets all four sides to have margin of 5 px;
- `margin: 10px 20px;`
 - ✓ top and bottom to 10px, left and right to 20px;
- `margin: 5px 3px 8px;`
 - ✓ top 5px, left/right 3px, bottom 8px
- `margin: 1px 3px 5px 7px;`
 - ✓ top, right, bottom, left (clockwise from top)
- Same for padding

The Box Model



Positioning

- ❖ position: defines the positioning of the element in the page content flow
- ❖ The value is one of:
 - static (default)
 - relative - relative position according to where the element would appear with **static position**
 - absolute - position according to the innermost positioned parent element
 - fixed - same as absolute, **but ignores page scrolling**

Float

- float: the element “floats” to one side
 - left: places the element on the left and following content on the right
 - right: places the element on the right and following content on the left
 - margins of floated elements do not collapse
 - floated inline elements can apply height

Clear

- clear
 - ✓ Sets the sides of the element where other floating elements are NOT allowed
 - ✓ Used to "drop" elements below floated ones or expand a container, which contains only floated children
 - ✓ Possible values: left, right, both
- Clearing floats
 - ✓ additional element (<div>) with a clear style

Opacity

- opacity: specifies the opacity of the element
 - Floating point number from 0 to 1
 - For old Mozilla browsers use `-moz-opacity`
 - For IE use `filter:alpha(opacity=value)` where value is from 0 to 100; also,

Visibility

- visibility
 - Determines whether the element is visible
 - **hidden**: element is not rendered, but still occupies place on the page (similar to `opacity:0`)
 - **visible**: element is rendered normally

Display

- **display:** controls the display of the element and the way it is rendered and if breaks should be placed before and after the element
 - **inline:** no breaks are placed before and after (is an inline element)
 - **block:** breaks are placed before AND after the element (<div> is a block element)
 - **none:** element is hidden and its dimensions are not used to calculate the surrounding elements rendering (differs from visibility: hidden!)

Overflow

- **overflow:** defines the behavior of **element when content needs more space** than you have specified by the `size` properties or for other reasons. Values:
 - ✓ `visible` (default) - content spills out of the element
 - ✓ `auto` - show scrollbars if needed
 - ✓ `scroll` - always show scrollbars
 - ✓ `hidden` - any content that cannot fit is clipped

Other CSS Properties

- cursor: specifies the look of the mouse cursor when placed over the element
 - Values: crosshair, help, pointer, progress, move, hair, col-resize, row-resize, text, wait, copy, drop, and others
- white-space - controls the line breaking of text. Value is one of:
 - nowrap - keeps the text on one line
 - normal (default) - browser decides whether to brake the lines if needed

Table Styling Properties

- Table Borders

```
table, th, td {  
    border: 1px solid;  
}
```

- Table width thight

- ```
table {
 width: 100%;
 height: 40%;
}
```



## Cont.

---

- CSS Table Alignment

```
td {
 text-align: center;
 vertical-align: bottom;
}
```



# CSS: Creating Navigation Bars

---

```
<ul id="navbar">
 Home
 About US
 contact US
 help
 login

```

- [Home](#)
- [About US](#)
- [contact US](#)
- [help](#)
- [login](#)



## Cont..

---

```
#navbar {
 list-style-type: none;
 margin: 0;
 padding: 0;
 width: 200px;
 background-color: #EEEEEEE;
}
```

[Home](#)  
[About US](#)  
[contact US](#)  
[help](#)  
[login](#)



## Cont..

---

```
ul#navbar li a {
 display: block;
 color: #000000;
 font-weight: bold;
 padding: 8px 16px;
 text-decoration: none;
}
```

Home

About US

contact US

help

login



## Cont..

---

```
ul#navbar li a:hover {
 background-color: orange;
 color: white;
}
```

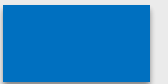
Home

About US

contact US

help

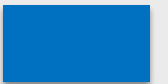
login



## Cont..

---

```
ul#navbar li {
 display: inline;
```



# CSS Measuring Units

- CSS Units
- CSS has several different units for expressing a length.
- Many CSS properties take "length" values, such as width, margin, padding, font-size, etc.

Unit	Description
cm	centimeters
mm	millimeters
in	inches (1in = 96px = 2.54cm)
px *	pixels (1px = 1/96th of 1in)
pt	points (1pt = 1/72 of 1in)
pc	picas (1pc = 12 pt)

## Benefits of using CSS

---

- More powerful formatting than using presentation tags
- Your pages load faster, because browsers cache the .css files
- Increased accessibility, because rules can be defined according given media
- Pages are easier to maintain and update



# End of Ch.5

---

Questions, Ambiguities, Doubts, ... ???

